arXiv:2404.18911v1 [cs.CL] 29 Apr 2024

Kangaroo: Lossless Self-Speculative Decoding via Double Early Exiting

Fangcheng Liu[†] Yehui Tang[†] Zhenhua Liu[†]
Yunsheng Ni[†] Kai Han[⋆,†] Yunhe Wang[⋆,†]

[†]Huawei Noah's Ark Lab [⋆] Corresponding Author

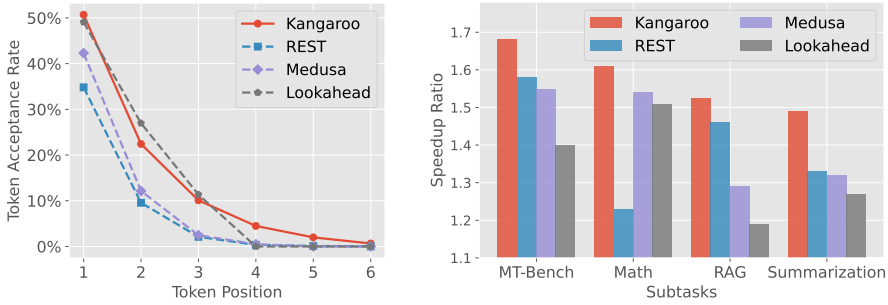
{liufangcheng3,kai.han,yunhe.wang}@huawei.com

Abstract

Speculative decoding has demonstrated its effectiveness in accelerating the inference of large language models while maintaining a consistent sampling distribution. However, the conventional approach of training a separate draft model to achieve a satisfactory token acceptance rate can be costly. Drawing inspiration from early exiting, we propose a novel self-speculative decoding framework *Kangaroo*, which uses a fixed shallow sub-network as a self-draft model, with the remaining layers serving as the larger target model. We train a lightweight and efficient adapter module on top of the sub-network to bridge the gap between the sub-network and the full model's representation ability. It is noteworthy that the inference latency of the self-draft model may no longer be negligible compared to the large model, necessitating strategies to increase the token acceptance rate while minimizing the drafting steps of the small model. To address this challenge, we introduce an additional early exiting mechanism for generating draft tokens. Specifically, we halt the small model's subsequent prediction during the drafting phase once the confidence level for the current token falls below a certain threshold. Extensive experiments on the Spec-Bench demonstrate the effectiveness of Kangaroo. Under single-sequence verification, Kangaroo achieves speedups up to $1.68\times$ on Spec-Bench, outperforming Medusa-1 with 88.7% fewer additional parameters (67M compared to 591M). The code for Kangaroo is available at https://github.com/Equationliu/Kangaroo.

1 Introduction

Large Language Models (LLMs) [1, 2, 3, 4, 5, 6] have undeniably showcased remarkable performance across a myriad of natural language tasks. However, constrained by the bottleneck of memory bandwidth [7], the primary latency



(a) The token acceptance rate on the *mathematical reasoning* subtask in Spec-Bench. Token position "2" represents the next-next-token prediction task.

(b) End-to-end speedup ratio comparison on four subtasks in Spec-Bench. "Math" and "RAG" denote mathematical reasoning and retrieval-augmented generation, respectively.

Figure 1: Comparison of various self-drafting speculative decoding methods on Spec-Bench [14] for Vicuna-7B [8]. Kangaroo outperforms all other methods *w.r.t.* end-to-end speedup ratio across all the four subtasks. For more detailed comparison on full Spec-Bench, see Table 1.

for autoregressive decoding of LLMs stems from memory read/write operations of model weights rather than arithmetic computations. For instance, decoding with Vicuna-33B [8] on four NVIDIA V100 GPUs yields a throughput of only seven new tokens per second. To address this challenge, Speculative Decoding (SD) techniques [9, 10] have been developed, aiming to accelerate autoregressive decoding by verifying multiple tokens generated by a draft model in parallel. Given γ draft tokens, SD can generate 1 to $\gamma + 1$ new tokens within each forward pass of the large LLM. The effectiveness of SD relies on two primary factors: 1) the gap between the draft model and the target LLM. Researchers often train a *tiny* draft model from scratch on a large corpus to accelerate large LLMs from the same series, *e.g.*, LLaMA-68M [11] for LLaMA-7B [2]. However, the training of such task-specific models can be costly [12, 13], limiting its application in real-world scenarios; 2) the inference latency of the draft model. If the inference cost of the small model is negligible compared to the target large LLM, the end-to-end speedup ratio is directly proportional to the consistent token acceptance rate as defined in Eq (2).

To address the aforementioned issues, several studies have proposed self-drafting methods that do not rely on external drafter models. LLMA [15] and REST [16] generate draft tokens by selecting text spans from reference or retrieving relevant tokens from the database. Notably, Medusa [17] trains multiple time-independent FFN heads on top of the last decoder layer. However, these approaches still present some challenges. While Medusa can efficiently generate multiple draft tokens at adjacent positions, its token acceptance rate is not yet satisfactory (see Figure 1(a)). Additionally, focusing exclusively on the token acceptance rate without considering the latency of generating draft

tokens can lead to suboptimal end-to-end acceleration. For instance, Lookahead [18] achieves a token acceptance rate comparable to Kangaroo in the mathematical reasoning subtask, significantly outperforming Medusa. However, due to its lower efficiency in generating draft tokens compared to Medusa, its end-to-end speedup ratio is slightly lower than that of Medusa (see Figure 1).

In response to these challenges, we design an autoregressive self-draft model by training a lightweight and efficient adapter module on top of a fixed shallow sub-network of the original large LLM. As shown in Figure 2, the adapter network architecture consists of only one multi-head attention [19] and two normalization layers [20]. Surprisingly, we find this simple design efficient but powerful, with only 11.3% of the parameters of the Medusa's heads[1]. To further reduce the inference latency of the self-draft model, we introduce an additional early exiting mechanism for generating draft tokens, aiming to avoid unnecessary costs on more difficult tokens.

To summarize, our main contributions are:

- We propose a novel self-speculative decoding framework based on a double early-exit mechanism, named Kangaroo. Firstly, the equivalent self-draft small model exits early from the fixed shallow layers of the large LLM and connects to an adapter network to generate draft tokens. Secondly, during the drafting phase, Kangaroo uses early exiting at suitable points to avoid unnecessary computational overhead on more challenging tokens.
- Kangaroo offers a low-cost approach to train a lightweight small model. Since the self-speculative draft model and the large LLM share some KV cache and computation, the only additional deployment requirement in practice is a small adapter network.
- Experiments on the Spec-Bench [14] validate the effectiveness of Kangaroo. Under single-sequence verification, Kangaroo achieves speedups up to $1.7\times$ on Spec-Bench, outperforming Medusa-1 with 88.7% fewer additional parameters, *i.e.*, 67M compared to 591M.

This paper is structured as follows: Section 2 reviews related works, and Section 3 introduces our framework, Kangaroo. The experimental section, Section 4, provides analysis and comparisons with various self-drafting methods, along with ablation studies to identify Kangaroo's key components. The conclusion is presented in Section 5.

[1]For detailed ablation studies on the architecture of the adapter, see Table 2.

2 Related work

Inference Acceleration of Large Language Models With the rapid development of large language models, significant research effort has been dedicated to accelerating their inference speed [21]. Techniques such as knowledge distillation [22], model compression [23] and quantization [24] have also been widely applied in this area. However, these approaches often require additional training of the backbone or substantial modifications to the model architecture. Recent efforts have explored early exiting on models like the T5 series [25, 26, 27] and decoder-only architectures [28]. However, since early exiting accelerates inference by saving subsequent computations, it inevitably incurs the issue of performance degradation [25].

Speculative Decoding Speculative Decoding (SD) has gained significant attention due to its ability to accelerate the inference of LLMs while maintaining the same sampling distribution. Generally, SD [9, 10] involves finding or training [12, 29] a small draft model closely aligned with the target LLM. Consequently, recent research has focused on more convenient self-drafting methods. For instance, approaches like blockwise parallel decoding [30] and Medusa [17] expedite the generation of draft tokens by training multiple time-independent Feedforward Neural Networks (FFNs) at the second-top-layer. Several self-drafting acceleration techniques are inspired by early exiting. Draft & Verify [31], for instance, generates draft tokens by skipping intermediate redundant layers of the target LLM. While this approach could achieve a high token acceptance rate, the inference latency of the "small model" is exceptionally high, which can hinder end-to-end acceleration efficiency. SPEED [32] adapts early exiting to pipelined speculative execution for transformer decoders that employ parameter sharing. Concurrently, we have learned that there are also several works [33, 34, 35] that make improvement on Medusa by introducing time dependency among the draft tokens. For more detailed summarization, we refer readers to a recent survey [14] on speculative decoding.

3 Kangaroo

In this section, we first delve into an in-depth analysis of token acceptance rate, compression rate, and speedup ratio for several self-drafting algorithms. Subsequently, we introduce our framework, Kangaroo, which employs self-speculative decoding by sharing a fixed shallow sub-network of the large LLM. To further reduce the inference latency of the self-draft model, we introduce an additional early exiting mechanism when generating draft tokens.

Notation. We use x^t to denote the discrete token sequence $(x_1, \cdots, x_t)$ and $x^{i:j}$ to represent sequence $(x_i, \cdots, x_j)$. Let $\mathcal{V}$ be a discrete space over all possible tokens in the LLM's vocabulary, we model the autoregressive process of

a language model $\mathcal{M}$ by the conditional distributions $\mathcal{M}(\cdot \mid x^t) \in \mathbb{R}^{|\mathcal{V}|}$ where $|\mathcal{V}|$ is the vocabulary size. We use subscript $\mathcal{M}_n(\cdot \mid x^t)$ to denote the n-th entry of the probability distribution. We denote the large target language model and the speculative small model as $\mathcal{M}^b$ and $\mathcal{M}^s$, respectively.

Token Acceptance Rate Decays along Speculative Direction Speculative decoding is often evaluated using two primary metrics: walltime speedup ratio and compression rate. Given a speculative decoding algorithm, we execute it to generate N new tokens and record the accepted tokens per forward of the large model as a list $S = [s_1, s_2, \cdots, s_{|S|}]$ where $\sum_k s_k = N$. The compression rate (CR) is defined as

$$\text{CR} = \frac{1}{|S|} \sum_k s_k. \tag{1}$$

Note that during the verification of speculative sampling, once a draft token is rejected by the large model $\mathcal{M}^b$, all subsequent tokens will be discarded regardless of their quality. Compression rate does not accurately reflect the acceptance levels of the drafting algorithm for tokens at varying distances. Thus, we propose a new evaluation metric called consistent token acceptance rate:

Definition 1. The *consistent token acceptance rate* $\text{CTAR}(w)$, given a prefix and a following window with size w, is the probability that the w guessed tokens from the draft model $\mathcal{M}^s$ are *all* accepted by the target model $\mathcal{M}^b$.

For the greedy decoding setting, $\text{CTAR}(x^t, w)$ is 0 if there is at least one inconsistent top-1 prediction between $\mathcal{M}^s$ and $\mathcal{M}^b$ within the window, otherwise 1. Similar to the compression rate, the consistent token acceptance rate could be calculate as:

$$\text{CTAR}(w) = \frac{1}{|S|} \sum_k \mathbb{I}(s_k - w > 0), \tag{2}$$

which is a decreasing function *w.r.t.* the window size w. We plot the empirical CTARs (for $w = 1, 2, \cdots, 6$) of several self-drafting speculative decoding algorithms on the mathematical reasoning subtask of Spec-Bench [14] in Figure 1(a). It can be seen that in addition to the token acceptance rate, the speed of generating draft tokens also has a significant impact on the final end-to-end speedup ratio.

3.1 Early Exiting as Self-Drafting Model

Training an additional small model from scratch is often costly, thus it is worth considering sharing a portion of the parameters with the target LLM. Inspired by the concept of early exiting, we directly extract hidden states from a fixed shallow sub-network of the target LLM and learn a mapping from the shallow layer to the final layer. Specifically, We train a lightweight and efficient adapter $\mathcal{A}$ to bridge the gap between the self-draft model $\mathcal{M}^s = \mathcal{A} \circ \mathcal{M}^b[: l]$ and the target model $\mathcal{M}^b$, where the early exit layer $l \in \{1, 2, \cdots, L\}$ and $\mathcal{A}$ denotes the

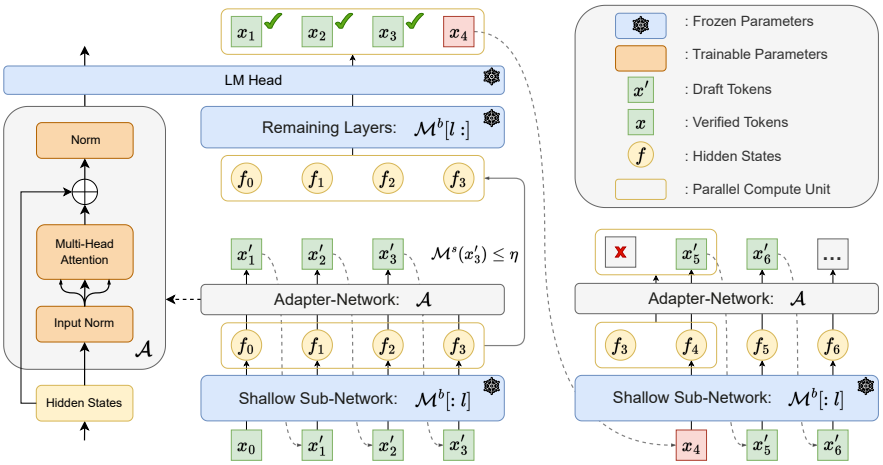


Figure 2: The framework of Kangaroo. The adapter network $\mathcal{A}$ consists of only one multi-head attention [19] and two normalization layers [20]. The self-draft model $\mathcal{M}^s = \mathcal{A} \circ \mathcal{M}^b[:l]$ will reuse the `LM Head` of the target LLM $\mathcal{M}^b$, where l denotes the early exit layer. To avoid unnecessary costs on more difficult tokens, $\mathcal{M}^s$ stops drafting once the confidence level of the current token falls below a certain threshold, *e.g.*, $\mathcal{M}^s(x_3') \leq \eta$. Note that we will concatenate the stopped token's *next early feature* f_3 with all previous exited features into a parallel compute unit $[f_0, f_1, \cdots, f_3]$, which will be verified by the remaining layers $\mathcal{M}^b[l:]$ in parallel. Once all drafted tokens are accepted ($x_i' = x_i$ for $i = 1, 2, 3$), we could start the next round with x_4 rather than x_3 if we have not calculated f_3 in advance. The decoding on parallel compute unit $[f_3, f_4]$ could save the latency for a single forward pass of the adapter network $\mathcal{A}$.

adapter network. As shown in Figure 2, the architecture of the adapter $\mathcal{A}$ consists of only one multi-head attention [19] and two normalization layers [20].

Training Loss A trivial method for training the adapter network is to maximize the token acceptance rate across each position, while we find that the cross-entropy loss exhibits faster convergence rate, *i.e.*,

$$\mathcal{A}^* = \arg\min_{\mathcal{A}} \sum_t \sum_n -\mathcal{M}_n^b(x_t) \log \mathcal{M}_n^s(x_t). \tag{3}$$

3.2 Dynamic Drafting Steps with Early-Exiting

Speculative decoding typically employs a fixed drafting step during the drafting phase, but this often leads to local optima. On one hand, the difficulty of

Table 1: Speedup comparison of various self-drafting speculative decoding methods on Spec-Bench [14] for Vicuna [8]. Speedup is the walltime speedup ratio and CR denotes the compression rate.

Size	Method	Translation		QA		Summarization		Math		RAG		MT Bench		Avg.
		CR	Speedup	CR	Speedup	CR	Speedup	CR	Speedup	CR	Speedup	CR	Speedup	
7B	Lookahead [18]	1.24	1.15×	1.56	1.21×	1.53	1.27×	1.96	1.51×	1.49	1.19×	1.70	1.40×	1.29×
7B	Medusa [17]	1.58	**1.41**×	1.50	1.34×	1.49	1.32×	1.73	1.54×	1.51	1.29×	1.76	1.55×	1.41×
7B	REST [16]	1.54	1.26×	1.91	**1.63**×	1.64	1.33×	1.53	1.23×	1.92	1.46×	2.00	1.58×	1.43×
7B	Kangaroo	1.41	1.24×	1.87	1.43×	1.87	**1.50**×	2.14	**1.61**×	2.05	**1.52**×	2.22	**1.68**×	**1.50**×
13B	Lookahead [18]	1.25	1.02×	1.39	0.99×	1.50	0.98×	1.94	1.24×	1.52	0.94×	1.68	1.08×	1.04×
13B	REST [16]	1.53	1.07×	1.92	**1.41**×	1.66	1.14×	1.55	1.06×	1.87	1.34×	1.98	1.36×	1.23×
13B	Medusa [17]	1.61	**1.33**×	1.49	1.25×	1.53	1.25×	1.80	1.48×	1.53	1.23×	1.82	1.48×	1.34×
13B	Kangaroo	1.45	1.18×	1.79	1.34×	2.00	**1.41**×	2.42	**1.63**×	2.16	**1.40**×	2.44	**1.66**×	**1.44**×

predicting the next token varies across different contextual scenarios. Therefore, it is highly likely to waste time on more challenging samples or miss opportunities to speculate on simpler tokens further. On the other hand, the inference of the small model used in this approach still incurs a certain cost, and timely termination can save a considerable amount of latency. Therefore, we stop drafting once the top-1 confidence on the self-draft model is below a predefined threshold η, *i.e.*,

$$\max_{n} \ \mathcal{M}_n^s(x) \leq \eta. \tag{4}$$

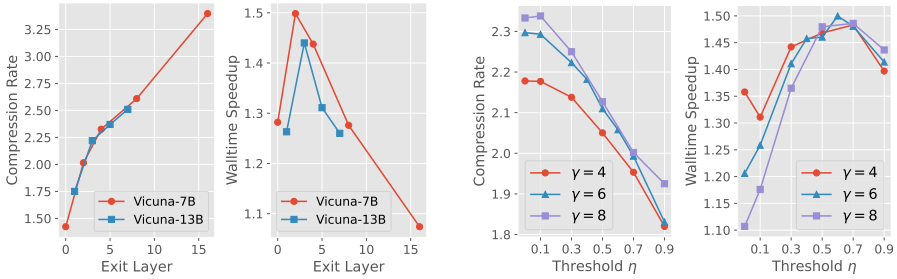
4 Experiments

4.1 Implementation Details

We conduct experiments on Vicuna [8] models with size of 7B and 13B. We select three self-drafting speculative decoding approaches for comparison, *i.e.*, Lookahead [18], Medusa [17] and REST [16]. We utilize the compression rate and the walltime speedup ratio metric. For fail comparison, we benchmark the performance of the selected self-drafting methods with the recently proposed Spec-Bench [14]. All models are evaluated on NVIDIA V100 GPUs. For Kangaroo, we train the adapter network for 10 epochs with the AdamW [36] optimizer on the ShareGPT dataset following Medusa [17].

4.2 Ablation Studies

The Depth of Shallow Sub-Network. The capacity of the self-draft model $\mathcal{M}^s$ highly depends on the depth of the shared shallow sub-network. However, selecting deeper early exiting layers, such as half layers of $\mathcal{M}^b$, would result in excessively high inference latency. Therefore, the the early exitlayer l controls a trade-off between token acceptance rate and drafting efficiency. As shown in Figure 3(a), we set $\ell = 2$ for Vicuna-7B and $\ell = 3$ for Vicuna-13B.



(a) Optimal exit-layer l.

(b) Optimal threshold η.

Figure 3: Ablation studies on hyper-parameters. The compression rate and walltime speedup is averaged across all sub-benchmarks in Spec-Bench.

The Architecture of the Adapter Module. In a transformer block, the FFN component counts for 67% of the whole parameters. As shown in Table 2, we find that removing the FFN component and sharing the `LM Head` of the target LLM is extremely effective.

Table 2: Ablation studies on the architecture of the adapter module $\mathcal{A}$ for Vicuna-7B. "Speedup" denotes the average speedup ratio on Spec-Bench [14].

Architecture	Input LN	Attention	Post LN	FFN	*Linear*	Last LN	Head	# Parameters	Speedup
Medusa	✗	✗	✗	✗	$\times 4$	✗	$\times 4$	591M	1.41 $\times$
Kangaroo	✓	✓	✗	✗	✗	✓	✗	**67M**	**1.50** $\times$
Kangaroo + Head	✓	✓	✗	✗	✗	✓	✓	198M	1.44 $\times$
1-Layer Transformer	✓	✓	✓	✓	✗	✓	✗	202M	1.37 $\times$
MLP Only	✓	✗	✗	✗	$\times 2$	✓	✓	165M	1.22 $\times$

Dynamic Exiting *v.s.* Fixed Step Drafting. To validate the effectiveness of our dynamic drafting steps with fixed threshold, we plot the comparison for various η in Figure 3(b). The fixed step strategy ($\eta = 0$) achieves the maximum compression rate, however, leading to sub-optimal end-to-end walltime speedup. Overall, the optimal threshold η is consistent across different maximum different steps. For Kangaroo, we set $\gamma = 6$ and $\eta = 0.6$.

5 Conclusion

In this paper, we introduced Kangaroo, a novel self-speculative decoding framework tailored for accelerating the inference of large language models. Kangaroo uses a fixed shallow sub-network to formulate a self-draft model, with the remaining layers serving as the larger target model. To reduce the inference latency of the self-draft model, we introduce an additional early exiting mechanism for generating draft tokens, aiming to avoid unnecessary costs on more difficult tokens. Under single-sequence verification, Kangaroo achieves speedups up to $1.7\times$ on Spec-Bench, outperforming Medusa-1 with 88.7% fewer additional parameters.

References

[1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.

[2] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

[3] Albert Q Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, et al. Mistral 7b. *arXiv preprint arXiv:2310.06825*, 2023.

[4] Yunhe Wang, Hanting Chen, Yehui Tang, Tianyu Guo, Kai Han, Ying Nie, Xutao Wang, Hailin Hu, Zheyuan Bai, Yun Wang, et al. Pangu-π: Enhancing language model architectures via nonlinearity compensation. *arXiv preprint arXiv:2312.17276*, 2023.

[5] Yehui Tang, Fangcheng Liu, Yunsheng Ni, Yuchuan Tian, Zheyuan Bai, Yi-Qi Hu, Sichao Liu, Shangling Jui, Kai Han, and Yunhe Wang. Rethinking optimization and architecture for tiny language models. *arXiv preprint arXiv:2402.02791*, 2024.

[6] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, Binyuan Hui, Luo Ji, Mei Li, Junyang Lin, Runji Lin, Dayiheng Liu, Gao Liu, Chengqiang Lu, Keming Lu, Jianxin Ma, Rui Men, Xingzhang Ren, Xuancheng Ren, Chuanqi Tan, Sinan Tan, Jianhong Tu, Peng Wang, Shijie Wang, Wei Wang, Shengguang Wu, Benfeng Xu, Jin Xu, An Yang, Hao Yang, Jian Yang, Shusheng Yang, Yang Yao, Bowen Yu, Hongyi Yuan, Zheng Yuan, Jianwei Zhang, Xingxuan Zhang, Yichang Zhang, Zhenru Zhang, Chang Zhou, Jingren Zhou, Xiaohuan Zhou, and Tianhang Zhu. Qwen technical report. *arXiv preprint arXiv:2309.16609*, 2023.

[7] Noam Shazeer. Fast transformer decoding: One write-head is all you need. *arXiv preprint arXiv:1911.02150*, 2019.

[8] Wei-Lin Chiang, Zhuohan Li, Zi Lin, Ying Sheng, Zhanghao Wu, Hao Zhang, Lianmin Zheng, Siyuan Zhuang, Yonghao Zhuang, Joseph E Gonzalez, et al. Vicuna: An open-source chatbot impressing gpt-4 with 90%* chatgpt quality. *See https://vicuna. lmsys. org (accessed 14 April 2023)*, 2(3):6, 2023.

[9] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023.

[10] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning*, pages 19274–19286. PMLR, 2023.

[11] Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Rae Ying Yee Wong, Zhuoming Chen, Daiyaan Arfeen, Reyna Abhyankar, and Zhihao Jia. Specinfer: Accelerating generative llm serving with speculative inference and token tree verification. *arXiv preprint arXiv:2305.09781*, 2023.

[12] Yongchao Zhou, Kaifeng Lyu, Ankit Singh Rawat, Aditya Krishna Menon, Afshin Rostamizadeh, Sanjiv Kumar, Jean-François Kagy, and Rishabh Agarwal. Distillspec: Improving speculative decoding via knowledge distillation. *arXiv preprint arXiv:2310.08461*, 2023.

[13] Sen Yang, Shujian Huang, Xinyu Dai, and Jiajun Chen. Multi-candidate speculative decoding. *arXiv preprint arXiv:2401.06706*, 2024.

[14] Heming Xia, Zhe Yang, Qingxiu Dong, Peiyi Wang, Yongqi Li, Tao Ge, Tianyu Liu, Wenjie Li, and Zhifang Sui. Unlocking efficiency in large language model inference: A comprehensive survey of speculative decoding. *arXiv preprint arXiv:2401.07851*, 2024.

[15] Nan Yang, Tao Ge, Liang Wang, Binxing Jiao, Daxin Jiang, Linjun Yang, Rangan Majumder, and Furu Wei. Inference with reference: Lossless acceleration of large language models. *arXiv preprint arXiv:2304.04487*, 2023.

[16] Zhenyu He, Zexuan Zhong, Tianle Cai, Jason D Lee, and Di He. Rest: Retrieval-based speculative decoding. *arXiv preprint arXiv:2311.08252*, 2023.

[17] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D Lee, Deming Chen, and Tri Dao. Medusa: Simple llm inference acceleration framework with multiple decoding heads. *arXiv preprint arXiv:2401.10774*, 2024.

[18] Yichao Fu, Peter Bailis, Ion Stoica, and Hao Zhang. Break the sequential dependency of llm inference using lookahead decoding. *arXiv preprint arXiv:2402.02057*, 2024.

[19] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

[20] Biao Zhang and Rico Sennrich. Root mean square layer normalization. *Advances in Neural Information Processing Systems*, 32, 2019.

[21] Zixuan Zhou, Xuefei Ning, Ke Hong, Tianyu Fu, Jiaming Xu, Shiyao Li, Yuming Lou, Luning Wang, Zhihang Yuan, Xiuhong Li, et al. A survey on efficient inference for large language models. *arXiv preprint arXiv:2404.14294*, 2024.

[22] Yuxian Gu, Li Dong, Furu Wei, and Minlie Huang. Minillm: Knowledge distillation of large language models. In *The Twelfth International Conference on Learning Representations*, 2023.

[23] Yehui Tang, Yunhe Wang, Jianyuan Guo, Zhijun Tu, Kai Han, Hailin Hu, and Dacheng Tao. A survey on transformer compression. *arXiv preprint arXiv:2402.05964*, 2024.

[24] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. Smoothquant: Accurate and efficient post-training quantization for large language models. In *International Conference on Machine Learning*, pages 38087–38099. PMLR, 2023.

[25] Tal Schuster, Adam Fisch, Jai Gupta, Mostafa Dehghani, Dara Bahri, Vinh Tran, Yi Tay, and Donald Metzler. Confident adaptive language modeling. *Advances in Neural Information Processing Systems*, 35:17456–17472, 2022.

[26] Sangmin Bae, Jongwoo Ko, Hwanjun Song, and Se-Young Yun. Fast and robust early-exiting framework for autoregressive language models with synchronized parallel decoding. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 5910–5924, 2023.

[27] Shengkun Tang, Yaqing Wang, Zhenglun Kong, Tianchi Zhang, Yao Li, Caiwen Ding, Yanzhi Wang, Yi Liang, and Dongkuan Xu. You need multiple exiting: Dynamic early exiting for accelerating unified vision language model. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 10781–10791, 2023.

[28] Neeraj Varshney, Agneet Chatterjee, Mihir Parmar, and Chitta Baral. Accelerating llama inference by enabling intermediate layer decoding via instruction tuning with lite. *arXiv e-prints*, pages arXiv–2310, 2023.

[29] Ziteng Sun, Ananda Theertha Suresh, Jae Hun Ro, Ahmad Beirami, Himanshu Jain, and Felix Yu. Spectr: Fast speculative decoding via optimal transport. *arXiv preprint arXiv:2310.15141*, 2023.

[30] Mitchell Stern, Noam Shazeer, and Jakob Uszkoreit. Blockwise parallel decoding for deep autoregressive models. *Advances in Neural Information Processing Systems*, 31, 2018.

[31] Jun Zhang, Jue Wang, Huan Li, Lidan Shou, Ke Chen, Gang Chen, and Sharad Mehrotra. Draft & verify: Lossless large language model acceleration via self-speculative decoding. *arXiv preprint arXiv:2309.08168*, 2023.

[32] Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Hasan Genc, Kurt Keutzer, Amir Gholami, and Sophia Shao. Speed: Speculative pipelined execution for efficient decoding. *arXiv preprint arXiv:2310.12072*, 2023.

[33] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. Eagle: Speculative sampling requires rethinking feature uncertainty. *arXiv preprint arXiv:2401.15077*, 2024.

[34] Zachary Ankner, Rishab Parthasarathy, Aniruddha Nrusimha, Christopher Rinard, Jonathan Ragan-Kelley, and William Brandon. Hydra: Sequentially-dependent draft heads for medusa decoding. *arXiv preprint arXiv:2402.05109*, 2024.

[35] Aonan Zhang, Chong Wang, Yi Wang, Xuanyu Zhang, and Yunfei Cheng. Recurrent drafter for fast speculative decoding in large language models. *arXiv preprint arXiv:2403.09919*, 2024.

[36] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017.